

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Description:

PocketSmart AI is a Generative AI-powered budget planning and recommendation assistant for everyday financial planning needs. It accepts a user budget together with contextual preferences and produces structured, budget-aware recommendations for Home Interior Planning, Party Planning, and Jewelry Planning. The system is designed to turn a broad budget into practical category-wise allocations, recommendations, and planning tips.

The application uses a FastAPI backend with Jinja2 templates, session-based authentication, a local database for users and recommendation history, a product catalog for example platform-oriented options, and Google Gemini integration for AI-generated recommendations. The AI prompt explicitly avoids claiming live stock, live availability, or exact current prices.

Scenarios

Scenario 1: A user enters a home budget, selects rooms such as Living Room or Bedroom, specifies quantities and a preferred style, and receives a practical allocation for furniture, lighting, decor, and essentials.

Scenario 2: A user enters a party budget, guest count, event type, venue, and theme. The system proposes a proportional plan for food, decoration, entertainment, gifts, and miscellaneous costs.

Scenario 3: A user enters a jewelry budget, occasion, style, outfit color and description, with optional outfit image input. The system returns occasion-aware jewelry suggestions and matching guidance.

Technical Architecture

Description: PocketSmart AI follows a modular web architecture. The browser-based frontend uses HTML, CSS, and Jinja2 templates. FastAPI receives validated requests, handles authentication/session state, calls the recommendation services, and stores recommendation history. The Gemini service generates structured JSON recommendations when the API is configured; fallback services provide basic recommendations when Gemini is unavailable.

Layer	Components	Responsibility
Frontend	Jinja2 templates, HTML, CSS, planner forms	Collect inputs and display recommendations
Backend	FastAPI, routes, Pydantic schemas	Validation, routing, sessions, APIs
AI Layer	Google Gemini via gemini_utils.py	Budget-aware structured recommendations
Data Layer	SQLite / DB module	Users and recommendation history
Catalog Layer	catalog.py and fallback.py	Example platform-oriented products and fallback plans

Request flow: User -> Jinja2 form -> FastAPI route -> Pydantic validation -> Gemini / fallback -> normalized JSON -> UI display -> history storage.

Database note: The project stores user accounts and recommendation history, therefore the data model includes users and recommendations tables with the recommendation record linked to the user.

Pre-requisites

- Python programming basics and virtual-environment usage.
- FastAPI and Pydantic concepts for API routing and validation.
- HTML, CSS, Jinja2 templates, and basic browser-side JavaScript.
- SQLite/database fundamentals for user and history storage.
- Google Gemini API access and safe environment-variable handling.
- VS Code and Uvicorn for local development and testing.

Project Workflow

- **Milestone 1:** Gemini AI setup and architecture planning.
- **Milestone 2:** Core planner functionality and authentication.
- **Milestone 3:** FastAPI backend integration and modular services.
- **Milestone 4:** UI development with Jinja2 templates.
- **Milestone 5:** Testing, fallback handling, and optimization.
- **Milestone 6:** Local execution, documentation, and demonstration preparation.

Milestone 1: Gemini AI Initialization and Architecture

This milestone establishes the core AI service and the application architecture. The Gemini client is created only when a Gemini API key is configured. The recommendation service asks the model to return valid JSON with planner, source, summary, budget allocation, recommendations, and tips.

Activity 1.1: Configure Gemini access

- Create the required Gemini API credential and keep it outside source control.
- Store the key in environment configuration used by config.py.
- Load GEMINI_MODEL from configuration so the model can be changed without rewriting business logic.

Activity 1.2: Define the AI response contract

- Require JSON output only.
- Use INR for recommendations unless a different currency is requested.
- Do not invent live listings, stock, exact current prices, or verified availability.
- Normalize missing recommendation fields before the result reaches the frontend.

Activity 1.3: Define module responsibilities

- routes/ handles HTTP endpoints.
- models/ contains structured input schemas.
- services/gemini_utils.py handles Gemini calls and normalization.
- services/fallback.py handles non-Gemini recommendations.
- services/catalog.py provides example recommendation data and platform links.

Activity 1.4: Project Configuration and Environment Setup

The application isolates secrets and runtime settings from the Python source files. The backend imports GEMINI_API_KEY and GEMINI_MODEL from config.py and initializes the Gemini client only when the key is present.

Example environment configuration

```
GEMINI_API_KEY=your_api_key_here  
GEMINI_MODEL=your_configured_gemini_model
```

Example local setup

```
python -m venv .venv  
.venv\Scripts\activate  
pip install -r requirements.txt  
uvicorn app.main:app --reload
```

Security practice: The environment file should not be committed with a real API key. Use an example configuration file for sharing the project.

Milestone 2: Core Functionalities Development

The core functionality is divided into three planners. Each planner validates its input, calls a dedicated recommendation service, and can save the result to the authenticated user history.

Home Interior Planner

- Accept a positive budget.
- Collect one or more rooms, quantities, preferred style, and location.
- Generate recommendations across furniture, lighting, decor, storage, and essentials.

Party Planner

- Accept a positive budget and guest count.
- Collect event type, venue, theme, and location.
- Generate plans covering food, decoration, entertainment, gifts, and miscellaneous costs.

Jewelry Planner

- Accept budget, occasion, style, outfit color, and description.
- Allow an optional outfit image upload.
- Generate occasion-aware jewelry recommendations with image-aware processing when supported.

Activity 2.3: Authentication and Session Management

PocketSmart AI provides registration, login, logout, bearer-token support, and session-based personalization. User credentials are validated and stored as password hashes rather than plain text.

Endpoint	Purpose
/register	Creates a new account and starts a session.
/login	Authenticates email and password and starts a session.
/logout	Clears the active session.
/token	Issues a bearer access token after successful authentication.
/session-info	Returns current login state and user information.
/session-data	Returns user data and recommendation count for the current session.
/history	Retrieves the user's recent recommendation history.

The history route is user-scoped, so a logged-in user can review prior Home, Party, or Jewelry recommendations saved for that account.

Milestone 3: Backend - FastAPI Integration

FastAPI acts as the central bridge between the frontend, validation models, recommendation services, authentication, and database layer. The project uses structured Pydantic models so the planners reject invalid budgets and malformed input before recommendation generation.

Activity 3.1: Define FastAPI routes

Route	Function
/generate-home	Generate Home Interior recommendations.
/generate-party	Generate Party Planning recommendations.
/generate-jewelry	Generate Jewelry recommendations, including optional image input.
/recommendations-details	Return category-specific catalog details for an authenticated user.
/startup	Expose configuration readiness information.
/health	Return service health status.

The backend can also return fallback recommendations if the Gemini API is missing or fails, allowing the application to continue producing a usable budget plan.

Activity 3.2: Structured Request Models

Model	Important inputs
HomeRequest	budget, rooms, quantities, style, location
PartyRequest	budget, guests, event_type, venue, theme, location
JewelryRequest	budget, occasion, style, outfit_color, outfit_description, optional outfit_image
RecommendationDetailsRequest	planner, category, budget

Validation includes positive budgets, required text fields, bounded string lengths, and guest-count limits. The Jewelry endpoint also validates that an uploaded file is an image and checks the configured maximum upload size.

Example Home request shape

```
{
  "budget": 25000,
  "rooms": ["Living Room", "Bedroom"],
  "quantities": {"lights": 4, "fans": 2},
  "style": "modern",
  "location": "India"
}
```

Activity 3.3: Gemini Integration and JSON Normalization

The Gemini service sends a system instruction that defines PocketSmart AI as a practical budget-planning assistant. The call requests JSON output and sets generation parameters such as temperature and maximum output tokens in the SDK configuration.

The service then parses the returned text. It removes optional markdown code fences, locates the JSON object, loads the JSON, and normalizes missing fields such as summary, source, budget allocation, tips, category, price, rating, discount, platform, unit, and reason.

Core response structure

```
{
  "planner": "home",
  "source": "gemini",
  "summary": "...",
  "budget_allocation": {...},
  "recommendations": [...],
  "tips": [...]
}
```

Activity 3.4: Fallback Recommendation Services

Fallback logic keeps the application useful when Gemini is not configured or a model call cannot be completed. Each planner has deterministic budget-allocation logic and a small recommendation set. These values are example recommendations, not proof of live availability or current market prices.

Planner	Example allocation categories
Home	Furniture, lighting, decor, storage/essentials
Party	Food, decoration, entertainment, gifts, miscellaneous
Jewelry	Necklace, earrings, bangles/bracelet, miscellaneous

This layered design separates the AI path from the fallback path so the UI can still render the same normalized response shape.

Activity 3.5: Product Catalog and Platform Links

The catalog service stores example items for common planning categories and associates each item with a platform label. The search function filters categories, sorts items by budget fit and rating, and adds a generated search URL for the associated platform.

Category	Example platform labels
Home	Amazon, IKEA, Flipkart
Party	Amazon, Flipkart, Swiggy, Zomato, OYO
Jewelry	Amazon, Flipkart

The application treats platform names as source categories. The AI instructions explicitly avoid claiming that an item is in stock or available at the displayed example price.

Milestone 4: UI Development

The frontend is implemented with Jinja2 templates and a shared CSS file. Page routes include the home page, login, register, dashboard, and individual planners for Home, Party, and Jewelry. The planner forms are designed to keep budget and context inputs visible and easy to understand.

Page	Purpose
Home	Introduces PocketSmart AI and links to planning modules.
Register	Collects name, email, and password for account creation.
Login	Authenticates an existing user.
Dashboard	Shows user-oriented content and saved activity.
Home Planner	Collects home budget, room and style details.
Party Planner	Collects event, guest, venue, theme, and budget details.
Jewelry Planner	Collects jewelry preferences and optional outfit image.

Activity 4.1: Home Interior Planner UI

The Home Interior Planner is designed for budget-based room planning. The user can select one or more rooms, add item quantities, choose a style such as modern, and specify a location.

The form maps directly to HomeRequest so the backend receives a structured object. The recommendation response is then presented as a summary, budget allocation, item suggestions, and practical tips.

Example input

- Budget: Rs. 25,000
- Rooms: Living Room, Bedroom
- Quantities: lights, fans, dining table
- Style: modern
- Location: India

The design keeps the user's stated budget visible throughout the planning process, supporting a clear connection between input and recommendations.

Activity 4.2: Party Planner UI

The Party Planner turns an overall event budget into category-wise planning. Required information includes budget, guest count, and event type, with optional venue and theme fields.

The service can structure recommendations around catering, decoration, entertainment, gifts, venue-related options, and miscellaneous expenses. Example platform categories include Swiggy, Zomato, OYO, Amazon, and Flipkart.

Example event

- Budget: Rs. 20,000
- Guests: 40
- Event: Birthday
- Venue: Home / flexible venue
- Theme: Elegant

Activity 4.3: Jewelry Planner UI

The Jewelry Planner supports text-based style matching and optional outfit image input. The form collects budget, occasion, style, outfit color, and outfit description. The backend validates the image MIME type and configured upload size before passing the image to the recommendation service.

Example input

- Budget: Rs. 5,000
- Occasion: Wedding reception
- Style: Elegant
- Outfit color: Maroon
- Optional outfit image: uploaded by the user

The response can include necklace, earrings, bangles/bracelet, and other matching recommendations, together with reasons and budget guidance.

Milestone 5: Testing and Optimization

Testing focuses on input validation, route behavior, authentication, AI response normalization, history persistence, fallback behavior, and the three planner user journeys. The application should be exercised with small, medium, and larger budgets and with different combinations of preferences.

Test area	Representative check
Authentication	Register, login, logout, invalid credentials
Home planner	Positive budget, one/multiple rooms, invalid zero budget
Party planner	Positive budget, guest count limits, required event type
Jewelry planner	Text-only request, image upload, invalid non-image file
Gemini path	Valid JSON response and missing API key behavior
Fallback path	Recommendations remain renderable when Gemini is unavailable
History	Authenticated results are saved and listed for the correct user

Activity 5.1: Reliability and Optimization

- Keep prompt output schema stable so the frontend can rely on predictable keys.
- Normalize incomplete AI records instead of allowing malformed results into the UI.
- Validate upload type and size before multimodal processing.
- Use fallback services for graceful degradation when Gemini is not configured.
- Store structured request/response JSON in recommendation history for later review.
- Keep environment-specific secrets outside source files.

Limitations to document during demonstration

- Catalog entries are example recommendation data and do not prove current price or stock.
- Live third-party inventory is not guaranteed by the platform labels.
- Local deployment is the current execution mode unless the application is separately hosted.
- AI quality depends on the configured Gemini model and API availability.

Milestone 6: Local Execution and Demonstration

Run Instructions

- Open the project in VS Code.
- Create and activate a Python virtual environment.
- Install dependencies from requirements.txt.
- Configure the Gemini API key and model in the environment configuration.
- Run the application with Uvicorn.
- Open <http://127.0.0.1:8000> in a browser.
- Register an account, log in, and test Home, Party, and Jewelry planners.
- Review the saved recommendation history from the authenticated session.

```
uvicorn app.main:app --reload
```

The FastAPI application exposes startup and health routes that can be used as simple readiness checks.

Project File and Folder Structure

```
PocketSmartAI/  
app/  
main.py  
config.py  
db.py  
security.py  
models.py  
dependencies.py  
routes/  
auth.py  
pages.py  
api.py  
services/  
auth.py  
catalog.py  
fallback.py  
gemini_utils.py  
models/  
schemas.py  
templates/  
home.html  
login.html  
register.html  
dashboard.html  
planner_name.html  
planner_party.html  
planner_jewelry.html  
history.html  
static/  
style.css  
requirements.txt  
README.md  
.env.example
```

The modular file structure separates routing, schemas, security, catalog data, fallback logic, AI integration, templates, and static assets. This separation supports easier maintenance and clearer project demonstration.

Conclusion

PocketSmart AI combines budget planning with Generative AI recommendations across three practical domains: Home Interior, Party Planning, and Jewelry Planning. The architecture separates user interface, FastAPI routing, validation, authentication, AI generation, catalog search, fallback logic, and data persistence.

The application is designed to be budget-aware and transparent about the limits of recommendation data. It can generate structured JSON recommendations through Gemini when configured and can fall back to deterministic examples when the AI service is unavailable. User-specific recommendation history provides continuity across sessions.

Future enhancement areas

- Connect live product or service APIs with explicit freshness indicators.
- Add richer analytics for spending patterns and planner usage.
- Improve recommendation explanations with user-editable constraints.
- Add cloud deployment and production-grade monitoring.
- Expand multilingual interaction and accessibility support.

Team: Nivetha, Shamini, Pooja Shri, Deepika, Devi Priya

Team ID: 6aba2e98ca93a2750963fe65